# Prédire les cas de COVID-19 à sept jours

Une chaîne MLOps complète, des données publiques jusqu'à une application conteneurisée : modèle versionné, promu par comparaison, testé en intégration continue, surveillé en production et réentraîné automatiquement.

RÉALISÉ PAR

Luvengani Dass
Saadia Szapiro

DONNÉES

Our World in Data
France · 2020 → 2023

MODÈLE

Random Forest
11 variables · horizon J+7

LIVRABLE

10 services Docker
API · MLflow · Airflow · Streamlit

# Anticiper une semaine à l’avance, avec ce que l’on sait aujourd’hui

- **La question posée.** À partir de l’historique connu le jour J, estimer le nombre de cas qui sera publié le jour J+7.
- **Pourquoi sept jours.** C’est le délai utile : une semaine suffit pour ajuster une capacité de dépistage, un planning hospitalier ou une campagne de rappel, et reste assez court pour que le passé récent porte de l’information.
- **La contrainte de rigueur.** Aucune variable postérieure au jour J ne doit entrer dans le modèle, sous peine de fuite temporelle — une erreur qui produit d’excellents scores et un modèle inutilisable.

**FORMULATION**

Régression supervisée sur série temporelle. Une ligne = un jour. La cible est la valeur observée sept jours plus tard.

**CE QUE CE N’EST PAS**

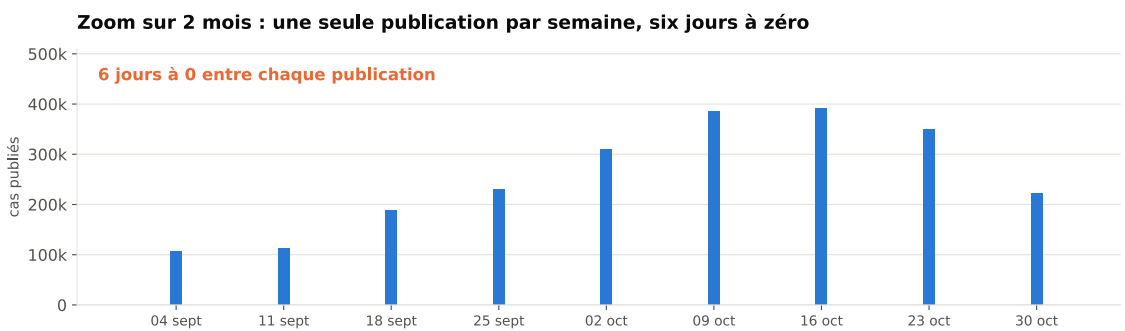
Pas un modèle épidémiologique compartimental (SIR, SEIR). Pas un modèle causal : il apprend des régularités statistiques, il n’explique pas la transmission.

# De 429 435 lignes mondiales à 1 387 jours et 11 variables

- **Collecte.** L'export public d'Our World in Data, téléchargé par l'étape `ingest` : six colonnes, filtrées sur la France.
- **Nettoyage.** Valeurs manquantes comblées par 0, cas négatifs ramenés à 0.
- **Variables.** Sept décalages, une moyenne mobile sur sept jours, trois variables de contexte. Aucune information postérieure au jour J.

LE CONSTAT QUI CHANGE LA LECTURE

La France ne publie qu'une fois par semaine : **88 % des journées valent zéro** (1 224 sur 1 387).




Septembre et octobre 2022 : une seule barre non nulle par semaine, entourée de six zéros.

# Une forêt aléatoire, qui bat les références naïves

**943** cas

MAE — ERREUR ABSOLUE MOYENNE

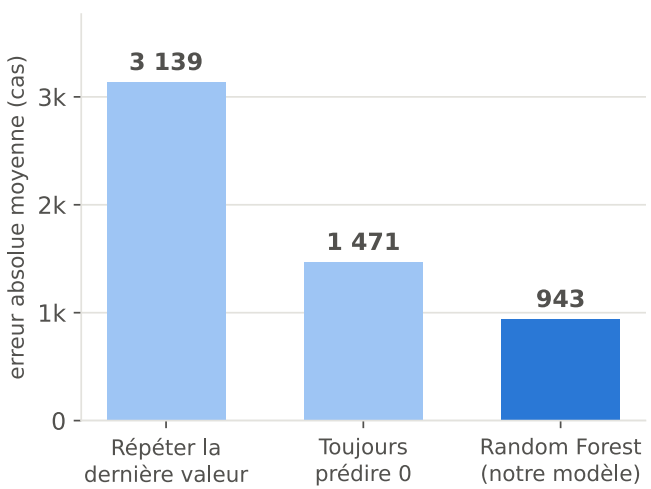
**4 262** cas

RMSE — ERREUR QUADRATIQUE

**0,68**

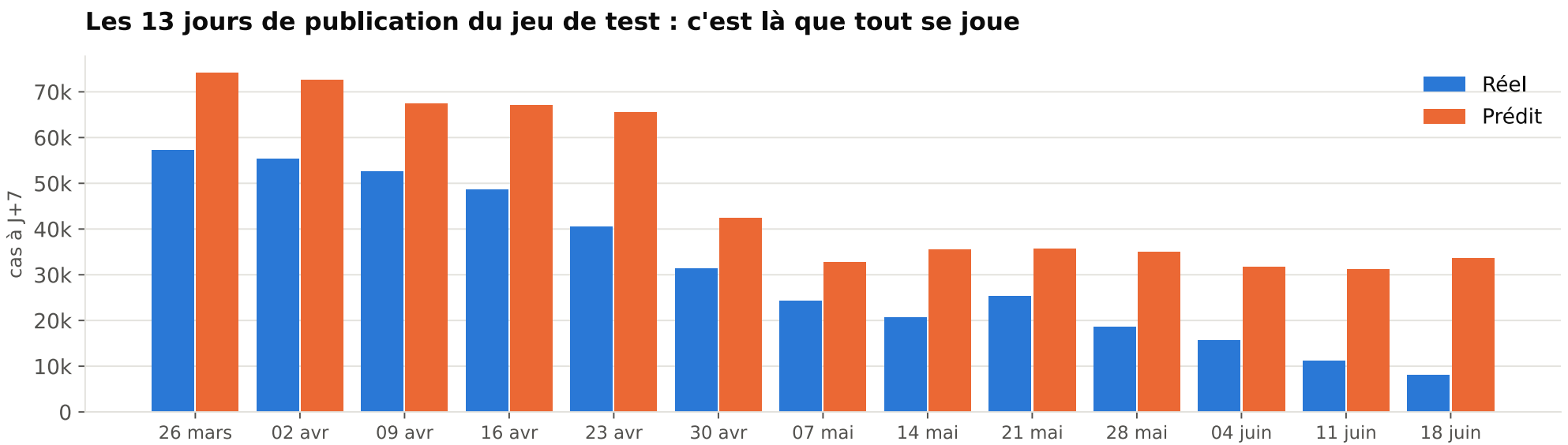
$R^2$ — VARIANCE EXPLIQUÉE

**MAE : le modèle face à deux références simples**




- **RandomForestRegressor**, 100 arbres, graine fixée : l'entraînement est reproductible.
- **Découpage chronologique**, jamais aléatoire : 1 109 jours d'apprentissage, 278 jours de test.
- **−70 % d'erreur** face à « répéter la dernière valeur », **−36 %** face à « toujours prédire zéro ».

# Une MAE flatteuse : toute l'erreur tient en 13 journées




**176** cas

MAE SUR LES 265 JOURS SANS PUBLICATION

**16576** cas

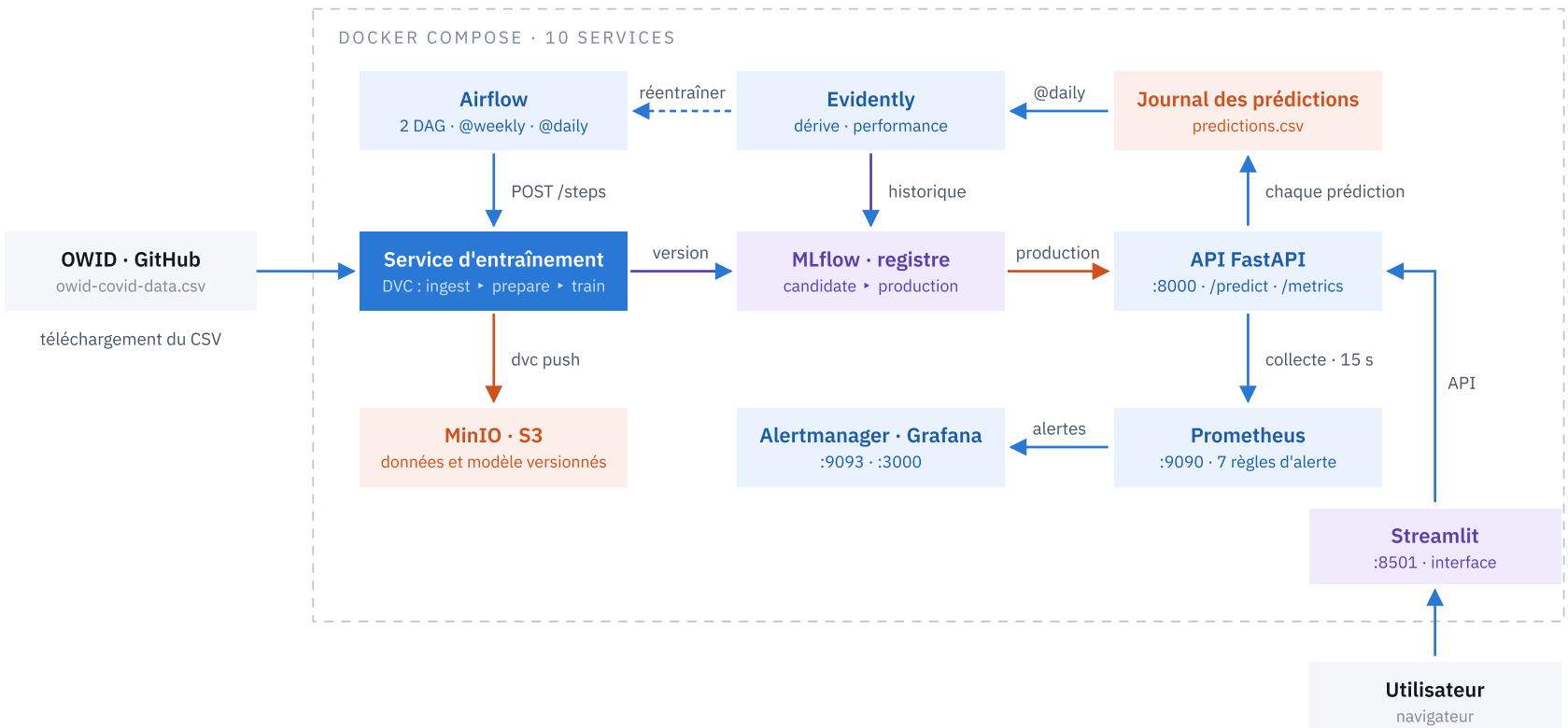
MAE SUR LES 13 JOURS DE PUBLICATION

**72,9** %

POIDS DE LAG_7 ; LAG_1 À LAG_6 PÈSENT 0 %

Le modèle a retrouvé seul la cadence hebdomadaire, mais il surestime en décrue. L'analyse complète est dans l'application Streamlit, sections Résultats et Interprétabilité.

# Dix services, trois boucles : entraîner, servir, surveiller




Un conteneur par rôle : l'API sert, le service d'entraînement calcule, Airflow orchestre, MLflow trace, Streamlit affiche. Une seule commande démarre l'ensemble : docker compose up.

# Une API d'inférence validée, protégée et testée

| ENDPOINT | RÔLE |
|---|---|
| `POST /predict` | prévision à J+7, journalisée |
| `POST /simulate` | prévision « et si », non journalisée |
| `GET /data/…` | données d'exploration pour Streamlit |
| `POST /lab/train` | essai d'entraînement |
| `POST /model/reload` | recharge la version en production |
| `GET /health · /metrics` | état du service · Prometheus |

Validation Pydantic des onze champs ; documentation Swagger générée sur `/docs`.

**SÉCURITÉ**

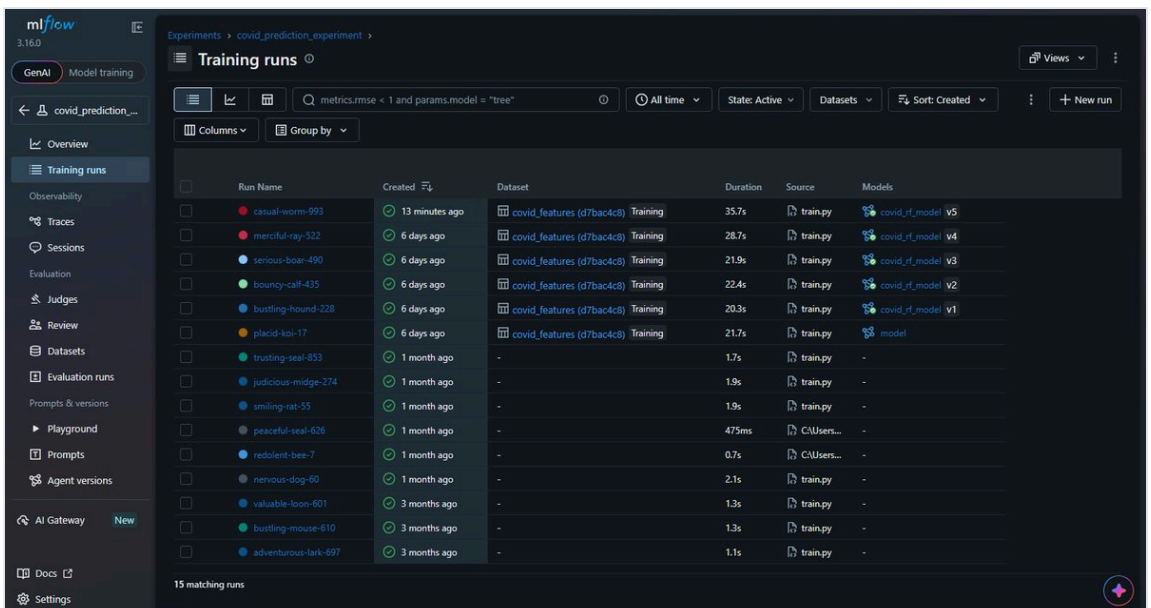
- La clé est lue dans `.env`, jamais écrite dans le dépôt.
- Sans clé, l'API **refuse de démarrer**.
- Clé absente ou incorrecte : `401`. La comparaison se fait en temps constant.

**MODÈLE SERVI**

- La version qui porte l'alias `production` dans le registre MLflow ; repli sur le fichier local si le registre est injoignable.

# Chaque entraînement laisse une trace comparable

- **9 paramètres** et **2 métriques** (MAE, RMSE) enregistrés à chaque run.
- Le **modèle**, le **jeu de données** utilisé et son empreinte.
- Les runs se comparent côte à côte, sans relire le code.
- Une seconde expérience, `covid_monitoring`, garde l'historique des contrôles de dérive.




Les runs de l'expérience : jeu de données, durée, et version du modèle enregistrée.

# À partir d'un modèle, retrouver ses données et son code

### DVC — LES DONNÉES ET LE MODÈLE

- Un pipeline en trois étapes : `ingest`, `prepare`, `train`.
- `dvc.lock` garde l'**empreinte** de chaque fichier, de chaque script et des paramètres : un commit git désigne une version exacte du code, des données et du modèle.
- `dvc repro` ne rejoue que ce qui a changé ; `dvc push` envoie les fichiers dans **MinIO**, un stockage compatible S3.

Retour arrière : `git checkout` puis `dvc checkout`.

### MLFLOW — LE LIEN ENTRE MODÈLE ET DONNÉES

- Chaque version du modèle porte l'empreinte DVC de ses données et le commit git.
- Git versionne le code, DVC les fichiers lourds, MLflow chaque entraînement.

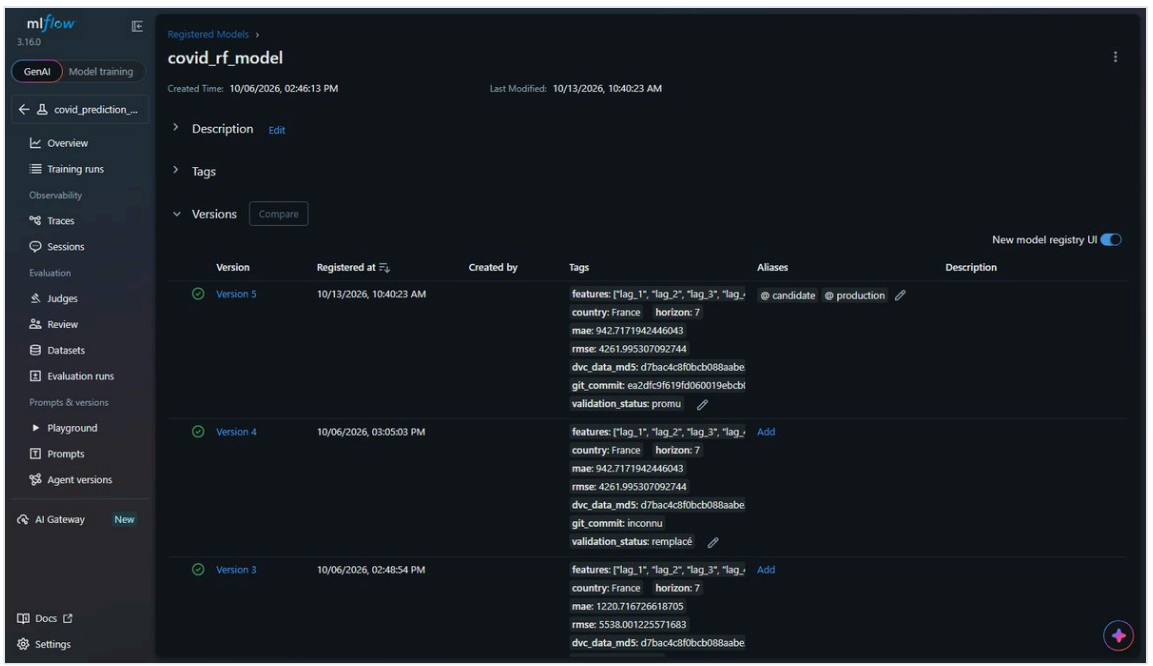
DERNIER ENTRAÎNEMENT

| | |
|---|---|
| Version du modèle | covid_rf_model v5 |
| Empreinte des données | d7bac4c8f0bc… |
| Run MLflow | 51437571920c… |

# Un modèle n’entre en production que s’il fait au moins aussi bien

- `candidate` : la dernière version entraînée, à valider.
- `production` : la version servie par l’API.
- Le candidat et la production sont évalués **sur le même jeu de test** ; le candidat ne passe que si sa MAE est inférieure ou égale.
- Chaque version porte son verdict : promu, rejeté, remplacé.

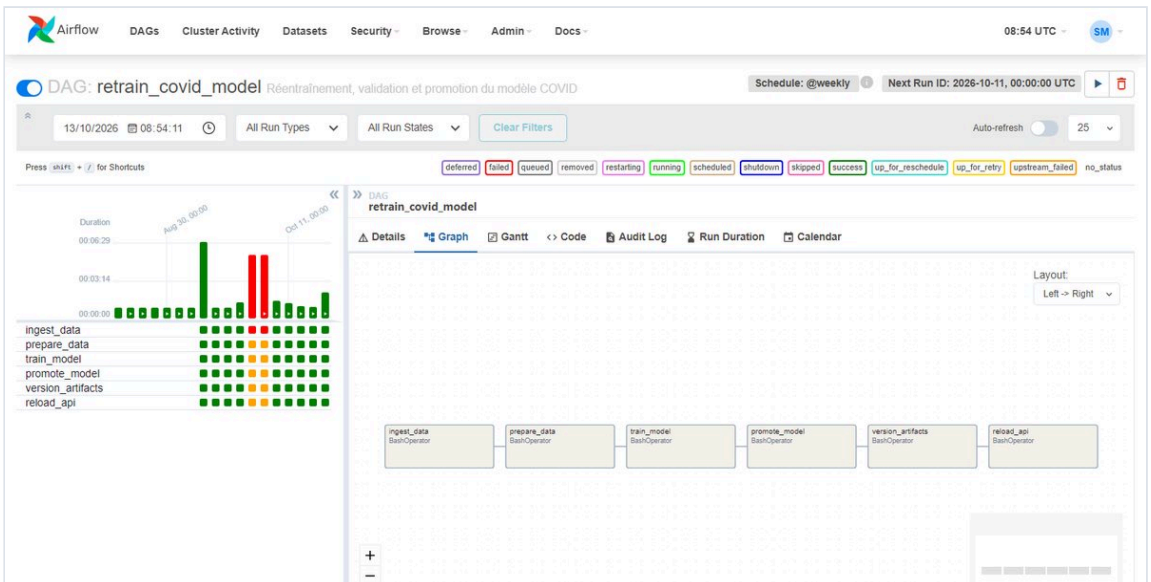
Retour arrière : déplacer l’alias `production`, puis appeler `POST /model/reload`. Les alias remplacent les étapes Staging / Production, dépréciées par MLflow.




Le registre : la version 5 est en production ; la version 3, moins bonne, a été rejetée.

# Airflow orchestre, le service d'entraînement calcule

- DAG `retrain_covid_model`, planifié `@weekly` : six tâches, de l'ingestion au rechargement de l'API.
- Airflow n'embarque aucune dépendance du projet : chaque tâche appelle le **service d'entraînement** (`POST /steps/…`).
- Une étape en échec renvoie une erreur : la tâche échoue, et le modèle en production reste en place.
- `reload_api` fait charger la nouvelle production, sans redémarrage.




Les six tâches du DAG, et à gauche l'historique de leurs exécutions.

# Chaque push est testé, chaque image est reconstruite

**25**
TESTS AUTOMATIQUES

**4**
IMAGES PUBLIÉES SUR GHCR.IO

**2**
WORKFLOWS

### TESTS.YML — À CHAQUE PUSH

- Un job teste l'API et le service d'entraînement.
- Un second teste les règles de dérive, avec Evidently.
- Le modèle n'est pas dans git : les tests en construisent un de remplacement.

### DOCKER.YML — À CHAQUE PUSH SUR MAIN

- Construit les images API, MLflow, entraînement et Streamlit.
- Les publie dans le registre de conteneurs GitHub.
- Deux étiquettes par image : `latest` et l'identifiant du commit.

# Le service sous surveillance

- L'API expose `/metrics`, collecté toutes les **15 secondes** : requêtes, temps de réponse, prédictions par version.
- **7 règles d'alerte** : API indisponible, aucun modèle chargé, taux d'erreur, latence, dérive, performance dégradée, monitoring en retard.
- **Alertmanager** regroupe les alertes et les achemine.

Le canal de notification (mail, Slack) reste à brancher.



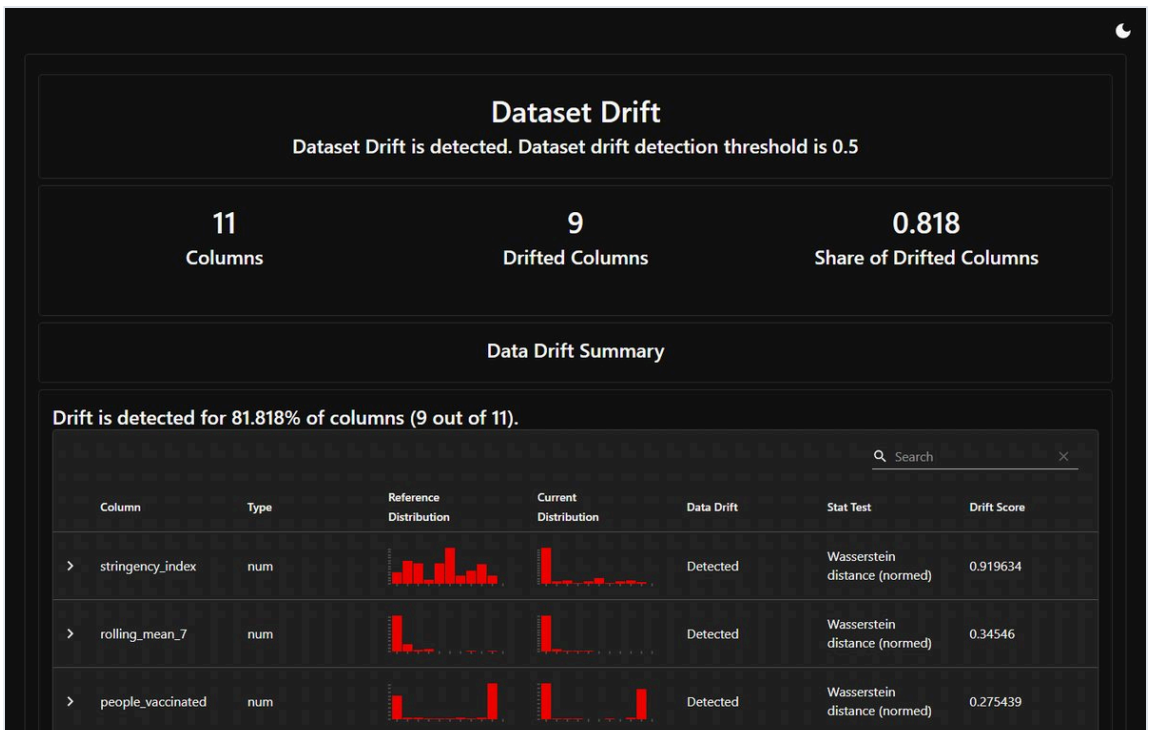

Le tableau de bord, créé au démarrage : état de l'API, version servie, dérive, trafic.

# Le modèle sous surveillance, et la réaction automatique

- Chaque prédiction est **journalisée** avec ses entrées.
- **Dérive** : alerte si au moins la moitié des variables ont changé de distribution par rapport à l'entraînement.
- **Performance** : les prévisions datées sont rapprochées de la valeur observée à J+7.
- Le DAG `monitor_covid_model`, `@daily`, **relance l'entraînement** si nécessaire.

DERNIER CONTRÔLE · 2026-10-13

| | |
|---|---|
| Prédictions analysées | 478 |
| Variables en dérive | 9 sur 11 |
| Réentraînement recommandé | oui |




Le rapport Evidently : distribution de référence et distribution reçue, variable par variable.

# Une interface qui ne parle qu'à l'API

- Huit sections : contexte, exploration des données, modèle, architecture, résultats, interprétabilité, simulateur, conclusion.
- **Aucun fichier lu** : les chiffres et les graphiques viennent de `/data/…`, les prévisions de `/simulate`.
- Les essais faits dans l'application ne sont pas journalisés : ils ne faussent pas le suivi de dérive.
- Un conteneur de plus dans la pile, démarré avec le reste.




Le simulateur : les onze entrées se règlent à la main, l'API répond en direct.

# La chaîne en fonctionnement, du démarrage à l'alerte

01 **`docker compose up -d`**
Les dix conteneurs démarrent ensemble.

02 **Streamlit sur `:8501`**
Le simulateur affiche « API connectée » et la version du modèle servi.

03 **Swagger sur `:8000/docs`**
`/predict` sans clé : 401. Avec la clé : 200, la prévision et la version.

04 **MLflow sur `:5000`**
Les runs, puis le registre et ses alias `candidate` et `production`.

05 **Airflow sur `:8080` → Trigger DAG**
`retrain_covid_model` : six tâches au vert, de l'ingestion à la promotion.

06 **Trafic de démonstration**
`simulate_traffic.py --scenario recent`, puis le DAG `monitor_covid_model`.

07 **Grafana sur `:3000`, Alertmanager sur `:9093`**
La dérive apparaît sur le tableau de bord, l'alerte est routée.

08 **GitHub, onglet Actions**
Les tests et la construction des images, relancés au dernier push.

PLAN DE REPLI

Si un service tarde à démarrer, les captures des diapositives 08 à 15 montrent les mêmes écrans, pris sur la pile en fonctionnement.

# Ce qui est vérifié, et ce qui ne l'est pas encore

**25 / 25**

TESTS PYTEST RÉUSSIS

**0**

ÉCHEC

| CE QUI EST TESTÉ | TESTS |
|---|---|
| API : santé, clé, prévision, métriques | 7 |
| Données d'exploration, simulation, essais | 8 |
| Service d'entraînement | 6 |
| Construction des variables | 2 |
| Règles de dérive et de performance | 2 |

Les tests de l'API s'exécutent contre l'application réelle via le `TestClient` de FastAPI. GitHub Actions relance l'ensemble à chaque push.

### CONTENEURISATION

Quatre images construites (API, MLflow, entraînement, Streamlit), six images publiques et dix services orchestrés par un seul `docker compose up`. Les volumes assurent la persistance : modèle, base MLflow, métadonnées Airflow, stockage MinIO.

### ANGLES MORTS ASSUMÉS

- Aucun test sur le téléchargement ni sur l'entraînement lui-même.
- Pas de test d'intégration bout en bout sur la chaîne de réentraînement.

# Six limites que nous assumons

01 **La cadence hebdomadaire n'est pas traitée.** Travailler sur une série « quotidienne » composée à 88 % de zéros rend les métriques globales flatteuses et peu informatives.

02 **Six variables sur onze sont inertes.** lag_1 à lag_6 ne portent aucune information : le modèle en supporte le coût sans aucun bénéfice.

03 **Surestimation systématique en décrue.** Sur les treize publications testées, l'erreur relative moyenne atteint 82 %.

04 **Tout tourne sur une seule machine.** MinIO, MLflow et Airflow partagent le même poste : ni sauvegarde distante, ni reprise si la machine tombe.

05 **L'authentification est minimale.** Une seule clé, partagée par tous les clients, sans expiration.

06 **Les alertes n'ont pas de destinataire.** Elles sont déclenchées, regroupées et routées, mais aucun canal (mail, Slack) n'est branché.

Ces limites ne sont pas des oublis : les trois premières viennent de l'analyse des résultats, les trois autres de choix faits pour tenir le projet sur un seul poste.

# La chaîne tient ; c’est le signal qu’il faut maintenant redresser

### SUR LA MODÉLISATION

- **Rééchantillonner en pas hebdomadaire.** Une série de 201 points denses remplacerait 1 387 points dont 88 % de zéros.
- **Évaluer sur les seuls jours de publication**, et publier systématiquement l’écart à une référence naïve.
- **Valider par TimeSeriesSplit** et comparer la forêt à d’autres modèles.

### SUR L’INDUSTRIALISATION

- **Brancher un canal de notification** sur Alertmanager, mail ou Slack.
- **Héberger le stockage et le registre** hors du poste ; pour DVC, seule l’adresse change.
- **Renforcer l’authentification** : un jeton par client, à durée limitée (OAuth2 et JWT).
- **Prolonger l’intégration continue** jusqu’au déploiement.

L’infrastructure MLOps est en place et fonctionne de bout en bout. Le travail restant porte désormais sur la donnée et sur l’évaluation — c’est-à-dire sur ce qui fait la valeur d’une prévision.